

# PA2 – Shakespeare Text Generation Using n-grams

Due: May 21, 2026 at 11:59 PM

## Overview

In this assignment you will implement n-gram language models and train them to generate text using Tiny Shakespeare. All graded tasks are marked with TODO comments in `PA2.ipynb`.

## Learning Goals

- Understand text preprocessing and tokenization for language modeling.
- Implement a unigram model based on word frequencies.
- Implement a bigram model for next-word prediction with one word of context.
- Evaluate language models using log probability and perplexity.
- Generate Shakespeare-like text from trained language models.

## Rules and Allowed Libraries

- Use only the packages already imported in the notebook.
- Do not use external language-modeling libraries to implement the TODOs.
- GenAI tools that generate output may be used in course assignments (not tests or exams) to support your learning, as long as you do so honestly through proper documentation, citation, and acknowledgement.

## Deliverables

- Submit `PA2.ipynb` with all TODOs in Parts 1–3 completed and the Question 4 written responses filled in.
- Submit the notebook to Gradescope as an `.ipynb` file, not as a PDF.
- Do not rename required functions or change function signatures.

## Point Distribution (100 total)

| Part                                                  | Points |
|-------------------------------------------------------|--------|
| Part 1: Text Preprocessing and Tokenization           | 10     |
| Part 2: Unigram Language Model                        | 40     |
| Part 3: Bigram Language Model                         | 45     |
| Question 4: Short Written Responses (manually graded) | 5      |

## Optional Extension

Part 5, the general n-gram language model, is optional and ungraded. It is included for extra practice with longer context windows and smoothing.

## Notes

- The autograded portion of PA2 is worth 95 points total.
- Question 4 is graded manually from the written responses in the notebook.
- Gradescope will show some autograder checks immediately after submission, and additional autograder checks also run during grading but are not shown in advance.
- Some helper functions and analysis cells are provided in the notebook.

## Submission Checklist

- All required TODOs in Parts 1–3 are implemented.
- Question 4 written responses are filled in.
- Notebook runs end-to-end without errors.
- You did not rename required functions or change function signatures.
- Your `.ipynb` file is uploaded to Gradescope and the autograder shows no submission errors.

## Tips

- Run each nearby check cell after finishing a TODO.
- Submit each part as you go, because later functions depend on earlier ones.
- Use small synthetic sentences to test probabilities before relying on the full Shakespeare dataset.